

JSON: JavaScript Object Notation

JSON or JavaScript Object Notation is a lightweight data format used to exchange data between the frontend and backend in web applications. **Douglas Crockford** was the first man who introduced and created the **JSON** format first in **2000**. (**.json extension**)

JSON Data Types:

- String: "Amit"
- Number: 25 (includes integers and floating-point numbers)
- Boolean: true, false
- Array: ["item1", "item2"]
- Object: {"key": "value"}
- Null: null

Syntax:

```
{  
  "key1": "value1",  
  "key2": "value2",  
  "key3": "value3"  
}
```

Data send :

Data is sent as a JSON string using [JSON.Stringify](#) from backend to frontend over HTTP.

Ex: "{ \"name\": \"Mohit\", \"age\": 30 }"

Features:

There are different features of JSON as listed below:

- It is a lightweight format that helps to optimize the performance of the web app.
- It is independent of different programming languages.
- It has a vast support from different programming languages.
- The **JSON** format is very easy to understand.
- It can represent the complex data structures.

- JSON does not support functions or undefined values.

JSON Disadvantages:

- There are some disadvantages of using **JSON**:
- It does not have any Document Type Definition.
- It is not good for large scale data products.
- There's no type definition of it.
- It is not safe to use with untrusted services and browsers as it does not have a solid security mechanism.
- It can output a complex data in case of large data set which may be difficult to understand.

JSON with Padding (JSONP):

It is an older technique used to bypass the browser's **Same-Origin Policy** before modern Cross-Origin Resource Sharing (CORS) became widely supported.

How JSONP works

Instead of returning plain JSON, the server wraps the JSON response inside a JavaScript function call.

Normal JSON

```
{  
  "name": "Ankita",  
  "role": "Data Engineer"  
}
```

JSONP

```
callback({  
  "name": "Ankita",  
  "role": "Data Engineer"  
});
```

Here, callback is a JavaScript function provided by the client.

Why was JSONP used?

- To retrieve data from a different domain.
- To work around browser same-origin restrictions before CORS existed.

JSON v/s XML

JSON and XML are the formats used to transfer data over the network. But there are some differences between them as listed in the below table:

JSON	XML
It stands for JavaScript Object Notation.	It stands for Extensible Markup Language.
It doesn't use any tag.	It uses end tag.
It allows to create arrays.	It does not have arrays.
JSON is simple and easy to read as well as write.	XML is a little complex as compare to JSON.
It does not have a robust and strong security mechanism.	It is much secure than JSON.

There are some similarities between **JSON** and **XML** as listed below:

- Both of them are used to transmit data to the network connection.
- Both are simple and human-readable.
- They are language independent.

- Both of them supports hierarchical and nested structure i.e. values within values or objects with object.
- They can be fetched by sending requests to the servers.

What is Serialization and Deserialization in JSON?

- **Serialization:** It is the term refer to the process of transforming an object to the string.
- **Deserialization:** It is the process of converting a string back into an object.

There are some alternatives available for JSON as listed below:

- XML (Extensible Markup Language)
- YAML
- BSON (Binary JSON)
- CBOR (Concise Binary Object Representation)

JSON FOR DE

Module 1

Why DE need to deal with json :

REST API – JSON – Kafka – spark -s3

Application – json event – kafka – streaming event – Data Lake

Why is JSON so common in Data Engineering?

JSON is a flexible, language-independent data format widely used by REST APIs, Kafka events, and cloud services. It supports nested objects and arrays, making it suitable for representing complex business data. In data engineering, we commonly parse, validate, flatten, and transform JSON into analytics-ready formats such as Parquet, Delta Lake, or relational tables

When the interviewer says: "The API returns an array of orders."

Ex: { "orders": [{...}, {...}, {...}] }

Python: Loop through the list (for order in orders)

PySpark: explode()

Pandas: json_normalize(record_path=...)

json.loads() vs json.load()

Function	Input	Output	Used When
json.loads()	JSON string	Python dict or list	API response, Kafka message, database column
json.load()	JSON file object	Python dict or list	Reading JSON from a file

Syntax :

json.loads() (String → Python Object)

```
import json
```

```
raw = '{"name":"John","age":25}'
```

```
record = json.loads(raw)
```

json.load() (File → Python Object)

```
import json
```

with open("employee.json") as f:

record = json.load(f)

Function	Input	Output	Memory Trick
json.loads()	JSON String	Python dict/list	String → Python
json.load()	JSON File	Python dict/list	File → Python
json.dumps()	Python dict/list	JSON String	Python → String
json.dump()	Python dict/list	JSON File	Python → File

Summary :

Reading JSON :

JSON String -----json.loads()-----> Python Object

JSON File -----json.load()-----> Python Object

Writing JSON:

Python Object -----json.dumps()-----> JSON String

Python Object -----json.dump()-----> JSON File

Module 2: Safe Nested Access

- ⌚ 5. Bracket access vs .get()

- **1. Bracket Access []**

Use when you're **100% sure** the key exists.

record["user"]["city"]

If "city" is missing: `KeyError`

2. .get()

Safer because it returns `None` instead of raising `KeyError`.

```
record.get("user").get("city")
```

If "city" is missing:

`None`

No crash.

3. .get() is not always safe

Example:

```
record = {  
    "user": None  
}
```

Then:

```
record.get("user").get("city")
```

becomes

```
None.get("city")
```

Output:

`AttributeError` - Because `None` is **not** a dictionary.

Ans :

[] raises a `KeyError` when a key is missing. `.get()` returns `None`, making it safer for optional fields. However, if an intermediate object is `None`, calling `.get()` on it raises an `AttributeError`. For deeply nested JSON, I use a `safe_get()` helper or validate intermediate objects before accessing nested fields.

Syntax :

```
safe_get(record, "user", "address", "city")  
if key is none it give output as unknown
```

Null vs Missing Key

A null value means the field exists in the schema but currently has no value. A missing key means the field wasn't sent at all, possibly due to schema evolution or an older producer. These cases may require different downstream handling, such as data quality checks or compatibility logic.

How do you distinguish between a null value and a missing key?

if "email" in record:

 if record["email"] is None:

 print("Email is NULL")

 else: print("Email field is MISSING")

Why use a **sentinel object** instead of None?

dict.get() returns None both when a key is missing and when the key exists with a null value. A sentinel object is a unique placeholder that lets me distinguish these two cases, which is important when handling schema evolution, optional fields, and data quality in ETL pipelines.

Example :

```
SENTINEL = object()
```

```
record.get(key, SENTINEL)
```

>Missing key

↓

SENTINEL

>Null value

↓

None

>Real value

↓

actual value

pd.json_normalize()

I use pd.json_normalize() to flatten nested JSON into a tabular format for analysis in Pandas. It's suitable for datasets that fit in memory. For large-scale distributed datasets, I'd use PySpark with explode() and nested column selection instead.

Convert json into table format .

record_path : record_path tells pd.json_normalize() which nested array should be expanded into rows. Each object inside that array becomes a separate row in the output DataFrame.

Ex:

```
data = [
    {
        "student": "Alice",
        "marks": [
            {"subject": "Math", "score": 90},
            {"subject": "Science", "score": 95}
        ]
    }
]
```

Code :

```
pd.json_normalize(
    data,
    record_path="marks"
)
```

o/p:

2 rows for math and science separately

meta=["company"] one more attribute with record_path to decide parent

ex: pd.json_normalize(
 data,
 record_path="employees",
 meta=["company"]) it will add company in all row output along with product.

Parameter	Purpose
record_path	Explodes a nested array into multiple rows
meta	Keeps parent fields and copies them into every exploded row

error = “ignore”

```
pd.json_normalize(  
    data,  
    record_path="marks",  
    meta=["student"],  
    errors="ignore"  
)
```

So for any record if student name is missing it will generally throw error as `keyError` but due to `errors=ignore` it will add `NaN` in missing field and not throw any error .

Module 5: PySpark

Function	Purpose
<code>spark.read.json()</code>	Read JSON
<code>schema()</code>	Apply explicit schema
<code>printSchema()</code>	View schema
<code>StructType</code>	Nested object
<code>ArrayType</code>	JSON array
<code>MapType</code>	Key-value object
<code>col()</code>	Access nested field
<code>explode()</code>	Array → Rows
<code>explode_outer()</code>	Preserve null arrays
<code>from_json()</code>	JSON string → Struct

Function	Purpose
<code>to_json()</code>	Struct → JSON string
<code>size()</code>	Array length
<code>element_at()</code>	Access map value
<code>select("a.b")</code>	Flatten nested struct

One-Line Revision

- **Kafka JSON** → Read stream → `from_json()` → `explode()` → Write target
- **REST API JSON** → Fetch JSON → Spark DataFrame → Flatten → Load
- **Clickstream JSON** → Streaming events → Window aggregations → Analytics
- **Schema Evolution** → Handle new/changed fields with nullable schemas and schema evolution
- **Performance Optimization** → Explicit schema, column pruning, early filtering, partitioning, broadcast joins
- **End-to-End Flow** → Ingest → Parse → Flatten → Validate → Transform → Optimize → Store → Report

Beginner Level

1. What is Nested JSON?

Answer:

Nested JSON is a JSON document where a field contains another JSON object or an array instead of a primitive value.

Example:

```
{  
  "id":101,  
  "address":{  
    "city":"Chicago",  
    "state":"IL"  
  }  
}
```

2. What are the different nested data types in PySpark?

- StructType
 - ArrayType
 - MapType
-

3. What is StructType?

Answer:

StructType represents a nested JSON object.

Example

```
{  
  "address":{  
    "city":"Chicago"  
  }  
}
```

Schema

address : struct

4. What is ArrayType?

Represents an array.

```
{  
  "skills":["Python","Spark","SQL"]  
}
```

Schema

skills : array<string>

5. What is MapType?

Represents dynamic key-value pairs.

```
{  
  "scores":{  
    "math":95,  
    "science":98  
  }  
}
```

Reading JSON

6. How do you read JSON in Spark?

```
df = spark.read.json(path)
```

7. How do you inspect nested schema?

```
df.printSchema()
```

8. Why should you avoid schema inference?

Because it

- scans files
- increases execution time
- may infer wrong datatype
- causes inconsistency

Production systems use explicit schema.

9. How do you define explicit schema?

```
schema = StructType([  
    StructField("id", IntegerType()),  
    StructField("name", StringType())  
])
```

Accessing Nested Fields

10. How do you access nested fields?

```
df.select("address.city")
```

or

```
col("address.city")
```

11. How do you access array elements?

```
col("skills")[0]
```

12. How do you access MapType values?

```
col("scores")["math"]
```

explode()

13. What does explode() do?

Converts each array element into a separate row.

Before

John

[Python,SQL]

After

John Python

John SQL

14. Difference between explode() and explode_outer()?

explode()

- skips null arrays

explode_outer()

- keeps null arrays
-

15. When do you use explode()?

Whenever a JSON array must become individual rows.

Example

Orders

↓

One row per order.

from_json()

16. Why is from_json() needed?

Kafka stores data as bytes/string.

from_json()

↓

JSON String

↓

StructType

17. Difference between `spark.read.json()` and `from_json()`

`spark.read.json()`

- Reads JSON files.

`from_json()`

- Parses JSON string columns.
-

18. Where is `from_json()` commonly used?

- Kafka
 - REST API
 - Streaming
 - CDC events
-

`to_json()`

19. Why use `to_json()`?

Convert StructType back to JSON string before writing to

- Kafka
 - REST APIs
 - Event hubs
-

Flattening

20. How do you flatten nested JSON?

`df.select(`

`"id",`

```
"address.city",  
"address.state"  
)
```

21. How do you flatten arrays?

explode()

22. How do you flatten multiple arrays?

Explode one array at a time.

Schema Evolution

23. What is schema evolution?

Adding/removing/changing columns without breaking the pipeline.

24. How do you handle changing JSON schema?

- Nullable fields
 - Versioned schema
 - Delta Auto Merge (Databricks)
 - Explicit schema updates
 - Validation before loading
-

Kafka

25. Explain Kafka JSON processing.

Pipeline

Kafka

↓

ReadStream



Cast value to String



from_json()



explode()



Transform



Delta

26. Why cast Kafka value to String?

Kafka stores messages as binary.

Need

```
col("value").cast("string")
```

before parsing.

Performance

27. How do you optimize nested JSON processing?

- Explicit schema
 - Select required columns
 - Filter early
 - Avoid unnecessary explode()
 - Broadcast joins
 - Partition properly
 - Cache reused DataFrames
-

28. Why is explicit schema faster?

Spark doesn't scan files to infer datatypes.

29. Why avoid unnecessary explode()?

explode()

↓

More rows

↓

More shuffle

↓

Longer execution

30. How do you optimize explode()?

- Filter before explode
- Select only required columns

- Explode only necessary arrays
-

Real Project

31. Explain a nested JSON project.

Sample Answer (60 seconds)

In one project, we received nested customer order JSON data from Kafka. The messages contained customer details along with an array of orders and nested shipping information. We defined an explicit StructType schema and used `from_json()` to parse the Kafka message. Then we used `explode()` to convert each order into a separate row, flattened nested fields such as customer ID and shipping address, applied data quality checks, removed duplicates, and stored the transformed data in Delta tables. Finally, the curated data was loaded into Snowflake for reporting. Using an explicit schema and selective column extraction reduced processing time and improved pipeline stability.

Scenario-Based Questions

32. You receive nested JSON with three levels of nesting. What would you do?

- Read JSON
 - Inspect schema
 - Define explicit schema
 - Flatten StructType fields
 - Explode arrays one by one
 - Apply transformations
 - Load target
-

33. JSON structure changes unexpectedly. What will you do?

- Validate incoming schema
- Handle nullable fields
- Log bad records
- Update schema
- Test before deployment

34. Nested array contains millions of records. How will you optimize?

- Filter early
 - Select required columns
 - Explode only needed arrays
 - Repartition if required
 - Cache reused DataFrames
 - Tune shuffle partitions
-

35. Difference between StructType, ArrayType, and MapType?

Type	Represents	Example
StructType	Nested object	Address
ArrayType	List of values/objects	Orders, Skills
MapType	Dynamic key-value pairs	Scores, Attributes

File Formats

If the interviewer says...	Think...
Simple tabular data	CSV
Nested or API data	JSON
Analytics/Data Lake	Parquet
Kafka/Streaming	Avro
Hive/Hadoop	ORC
Databricks/Lakehouse	Delta Lake
Legacy enterprise systems	XML
Logs	TXT / NDJSON
Business reports	Excel

Rule to remember for interviews:

- **Fastest for analytics → Parquet**
- **Fastest for streaming → Avro**
- **Most feature-rich for lakehouses → Delta Lake**

Feature	CSV	JSON	Parquet	Avro	ORC	Delta Lake
Storage Type	Row	Row (Nested)	Columnar	Row	Columnar	Columnar (Parquet)
Schema	No	Optional	Embedded	Embedded	Embedded	Embedded
Supports Nested Data	✗	✓	✓	✓	✓	✓
Compression	✗	Limited	Excellent	Good	Excellent	Excellent
Read Speed	Slow	Slow	Very Fast	Medium	Very Fast	Very Fast
Write Speed	Fast	Medium	Medium	Fast	Medium	Medium
File Size	Large	Large	Small	Medium	Small	Small
ACID Transactions	✗	✗	✗	✗	✗	✓
Schema Evolution	✗	Limited	✓	✓	✓	✓
Best For	Data Exchange	APIs	Analytics	Streaming	Hadoop	Data Lakehouse

When to Use Which Format?

Use Case	Best Format
Excel export	CSV
REST API response	JSON
Kafka messages	Avro / JSON
Spark analytics	Parquet

Use Case	Best Format
Snowflake external tables	Parquet
Databricks	Delta
Hadoop Hive	ORC
Data Lake	Parquet / Delta
ML datasets	Parquet
Streaming	Avro

Text (Human Readable)

CSV

JSON

XML

TXT

TSV

Binary (Machine Readable)

Parquet

Avro

ORC

Delta (Parquet files)

Excel

File Format	Human Readable?	Can Open in Notepad?	Description
CSV	✓ Yes	✓ Yes	Plain text with comma-separated values
JSON	✓ Yes	✓ Yes	Plain text with key-value pairs
XML	✓ Yes	✓ Yes	Plain text with tags
TXT	✓ Yes	✓ Yes	Plain text
TSV	✓ Yes	✓ Yes	Plain text with tab-separated values
Avro	✗ No	✗ No	Binary format with embedded schema
Parquet	✗ No	✗ No	Binary columnar storage format
ORC	✗ No	✗ No	Binary columnar format for Hadoop/Hive
Delta Lake	✗ No*	✗ No	Stores Parquet (binary) data files plus a transaction log (the <code>_delta_log</code> metadata files are JSON and human-readable)
Excel (.xlsx)	✗ No	✗ No	Binary/ZIP-based Office document